Design Concepts

1. Abstraction

What it means:
Abstraction means **hiding unnecessary details** and **showing only what's important**.

Why:
It helps you **manage complexity** by focusing on **what something does**, not *how* it does it.

Example:
When you use a **"Login" button**, you don't care about the underlying code for authentication — you just see *Login*. The messy details are hidden.

In coding:
You might have a `sendEmail()` method the user of the method only needs to know *what* it does, not *how* it connects to the server.

Key Point: Abstraction makes systems easier to understand and change.

2. Architecture

What it means:
Architecture is the **big picture structure** of the whole system — how it's **divided into major components**, how these parts **communicate**, and how the system is **organized**.

Why:
It helps you plan **how to build large, complex software** in a manageable way.

Example:
Your **FitTrack** app architecture might have:

- **Frontend (UI)**: Android screens.
- **Backend**: database (SQLite or Firebase).
- **API layer**: to connect data to the UI.
- **Services**: background step counter, notifications.

Key Point: Architecture is like a building's blueprint — it decides how parts fit together.

3. Patterns

What it means:
A design pattern is a **proven solution** to a **common software problem**.

Why:
Instead of solving the same problem from scratch, you reuse a **standard approach**.

Example:

- **Singleton Pattern:** Ensure there is only **one instance** of a class (e.g., a single DB connection).
- **Observer Pattern:** Notify multiple parts when data changes (like updating UI when steps count updates).
- **MVC Pattern (Model-View-Controller):** Separate data (Model), UI (View), and logic (Controller).

Key Point: Patterns save time, avoid errors, and make code reusable and understandable.

4. Separation of Concerns

What it means:
Break the software into **distinct parts**, each with a **clear responsibility**.

Why:
Makes it easier to build, test, debug, and change.

Example:

- The **UI layer** should only handle how things look.
- The **Business Logic layer** should handle calculations (like BMI).
- The **Database layer** should only store/retrieve data.

In your app, the **step counting logic** shouldn't know about **how the screen looks** — that's a separate concern.

Key Point: Each part focuses on *one thing only* — no unnecessary mixing.

5. Modularity

What it means:
Divide the system into **small, self-contained modules** that can be developed, tested, and understood **independently**.

Why:
Makes software easier to build, test, maintain, and reuse.

Example:
Your FitTrack has separate modules:

- Step Counter Module
- Workout Module
- Water Reminder Module
- Nutrition Module

Each can be coded, tested, or even replaced **independently**.

Key Point: Smaller pieces are easier to handle than one giant block.

6. Functional Independence

What it means:
Each module should do **one specific task** and have **minimal interaction** with other modules.

Why:
Changes in one module won't break others.

Example:
The Water Reminder module shouldn't care how the Step Counter works — they share data if needed, but their core logic is separate.

Key Point: Makes debugging and upgrading easy — change one part without messing up others.

7. Refinement

What it means:
Start with a **big, general design** and break it down step-by-step into **smaller, more detailed parts**.

Why:
Helps you go from **abstract ideas to real code** in manageable steps.

Example:

- High-level: "The app will track steps."
- Next: "Use a foreground service for step counting."
- Next: "Use SensorManager to access step counter sensor."

- Next: Write the actual methods for listening to steps.

Key Point: Refinement = start broad, go detailed progressively.

8. Aspects

What it means:
Aspects are **cross-cutting concerns** .Parts of the system that affect many modules but don't belong to just one.

Why:
Helps manage **things that overlap multiple modules**.

Example:

- **Logging:** Keeping logs of user actions.
- **Security:** Checking user permissions.
- **Error handling:** Handling app crashes gracefully.

These aspects **touch multiple modules** but aren't the main job of any single module.

Key Point: Use Aspect-Oriented Programming (AOP) to handle these neatly.

9. Refactoring

What it means:
Improving **internal code structure** without changing its **external behavior**.

Why:
Makes code cleaner, faster, and easier to maintain.

Example:
Your step counting method works, but it's messy — you split it into smaller helper methods, rename confusing variables, remove duplicate code.

Key Point: Refactoring keeps your code **healthy and maintainable**.

10. Design Classes

What it means:
Design classes are the **blueprints** for objects in your software.

Classes can be of different **types**, each with a clear role:

- **Entity Classes:** Represent real-world things. E.g., User, StepRecord.
- **Boundary Classes:** Handle interaction between system and users. E.g., LoginScreen.
- **Control Classes:** Manage the flow or logic. E.g., GoalManager.

Why:
Good class design keeps the system organized, reusable, and easy to expand.

Example:
FitTrack might have:

- User class: holds user profile data.
- StepCounterService class: handles step detection logic.
- DatabaseHelper class: interacts with SQLite.
- GoalInputBottomSheet class: handles user input.

Key Point: Classes are the building blocks — good design = clear roles.